

AI Coding Workshop @ Raiffeisen

- **AI Coding** hilft uns nur dann schneller zu werden, wenn Vertraulichkeit, freigegebene Tools, Quality Gates und menschliches Review nicht verhandelbar sind.

Folie 1 — Agenda

- **1 Min. — Warum:** AI Coding kann Reibung reduzieren, aber nur auf einer gesunden Engineering-Basis.
- **3 Min. — Raiffeisen-Leitplanken:** Source-Code-Klassifizierung, freigegebene Enterprise-Tools, Security Review, MCP-Freigaben.
- **4 Min. — Readiness:** Architektur, Tests, Dokumentation, Isolation, Review-Kompetenz.
- **4 Min. — Arbeitsmodell:** Agent als Pair Programmer; zuerst planen, Kontext bewusst steuern, spec-driven arbeiten.
- **2 Min. — Tooling:** Copilot-Modi, Instructions, Skills, MCP mit Least Privilege.
- **1 Min. — Call to Action:** ein Repo bewerten und den nächsten sicheren Pilot definieren.

Folie 2 — Regel #1: Vertraulichkeit

- **Source Code ist mindestens „Intern“.** Wenn die Anwendung höher klassifizierte Daten verarbeitet, übernehmen Code und alle Kopien die höchste Klassifizierung.
- **Keine vertraulichen oder geheimen Code-/Dateninhalte in externe AI-Tools.** Nur freigegebene Enterprise-Instanzen und freigegebene Use Cases verwenden.
- **SITAN und Tool-Bedingungen vor der Nutzung prüfen.** Klären, wo Daten gespeichert werden, ob sie für Training genutzt werden und wer Zugriff hat.
- **MCP und Agents brauchen explizite Governance.** Zielsystem, Datenklassifizierung, vertrauenswürdige/nicht vertrauenswürdige Quellen, invasive Aktionen und Authentifizierungsmodell bewerten.
- **Die menschliche Verantwortung bleibt bei uns.** AI-Output muss reviewed, getestet und nachvollziehbar sein, bevor er in einen Branch oder PR gelangt.

Quellen für diese Optimierung: „AI for Code“ Governance, „AI & Security“, „Anträge MCP Server im Kontext AI for Code“ und Generative AI @ Raiffeisen.

Folie 3 — AI-Readiness: bevor der Agent das Repo berührt

Damit AI im Repo Wert liefert, muss die Basis stimmen:

1. **Saubere Architektur:** klare Modulgrenzen, explizite Verträge, wenig versteckte Kopplung.
2. **Automatisierte Quality Gates:** Linting, Tests, lokales SonarQube wo verfügbar, Architektur- und Clean-Code-Checks.
3. **Schneller Feedback-Loop:** Builds und Tests müssen schnell genug für sichere Iterationen sein.
4. **Kontext by Design:** README, ADRs, Runbooks, `copilot-instructions.md`, `AGENTS.md` oder `CLAUDE.md`.
5. **Security-Isolation:** containerisierte Ausführung für riskante Runs, keine Secrets oder Produktionskonfigurationen im Kontext, Tools nach Least Privilege.
6. **Team-Fähigkeit:** Entwicklerinnen und Entwickler können AI-Output hinterfragen, Diffs reviewen und Verhalten verifizieren.

⚠️ Wenn Readiness-Punkte 1–3 schwach sind, verstärkt AI bestehende Probleme: unklare Grenzen, flaky Tests und langsame Builds werden zu schnellerem Chaos.

Folie 4 — Wie ich einen AI-Agenten sehe

Analogien, die im Team funktionieren:

- 🧑 **Helfer** — schneller, effizienter Mitarbeiter
- 🔍 **Discoverer** — findet sich im Projekt zurecht, weiss oft Sachen, die ich noch nicht gemacht habe
- 👤 **Pair Programmer** — ich bleibe der Reviewer
- ⚙️ **Executor** — macht Dinge, die ich selbst noch nie umgesetzt habe

Wichtig: Es ist ein Kollege, kein Orakel — Verantwortung bleibt bei mir.

Folie 5 — Risiken & typische Entwickler-Sorgen

Risiken

- „**Almost right**“-**Frust:** AI-Output sieht plausibel aus, braucht aber Nacharbeit.
- **Debugging-Aufwand:** schneller generierter Code ist nicht automatisch schneller ausgeliefert.
- **Quality Drift:** ohne Architektur- und Testleitplanken entstehen technische Schulden.
- **Data Leakage:** Prompts, Logs, Screenshots oder Testdaten können geschützte Informationen enthalten.
- **Kosteneskalation:** lange Sessions, grosse Context Windows und Wiederholungen verbrauchen Tokens.

Konkrete Herausforderungen

- **Mitigation:** in *Ask* oder *Plan* starten; *Autopilot* nur für klar begrenzte Low-Risk-Tasks.
-

- **Mitigation:** Secrets, `*.env` , Produktionskonfigurationen und vertrauliche Daten nie in den Agent-Kontext geben.
- **Mitigation:** Diffs, Tests, Architektur-Impact und Security-Aspekte vor jedem Merge prüfen.

Folie 6 — LLM-Grundlagen in 3 Begriffen

- **Tokens** — Text wird in ~4-Zeichen-Stücke zerlegt; alles zählt (Prompt, Antwort, Files).
- **Context Window** — der "Kurzzeitspeicher" des Modells. Voll = alt wird vergessen.
- **Training Cutoff** — Modell kennt nur die Welt bis Datum X. Für aktuelle Infos → Context liefern (z. B. Docs) oder Tools (MCP, Web-Fetch).

Folie 7 — GitHub Copilot & Claude Code: Tooling einordnen

- **Inline Suggestions:** assistierend im Editor, gut für kleine Ergänzungen.
- **Agent Mode im IDE:** plant und ändert mehrere Dateien innerhalb von VS Code oder JetBrains.
- **Copilot CLI:** Terminal-Agent, kann Dateien lesen, ändern und Kommandos ausführen.
- **Claude Code:** ähnliche Agentic-Coding-Idee mit starkem Fokus auf `CLAUDE.md` , Regeln, Skills und Subagents.

Folie 8 — Copilot CLI Modi: sicher starten

- **Ask/Execute:** sicherer Standard für Erklärungen, Impact-Analyse und erste Exploration.
- **Plan:** Agent erstellt zuerst Plan und Rückfragen; ideal für Workshops und Team-Adoption.
- **Autopilot:** nur für klar abgegrenzte Low-Risk-Änderungen mit Allow-/Deny-Listen, Testkommando und Rollback-Pfad.

Guardrails via `permissions-config.json` :

- Allowlist erlaubter Shell-Kommandos
- Denylist für Files (`*.env` , Secrets, prod-Configs)

Folie 9 — Project Instructions: Regeln einmal definieren

Ebene	Datei
User (global)	<code>~/.copilot/copilot-instructions.md</code>
Project	<code>.github/copilot-instructions.md</code> , <code>AGENTS.md</code> , <code>CLAUDE.md</code>

Sprach-/Bereich-spezifisch

`.github/instructions/java.instructions.md` (mit `applyTo: Glob`)

Beispiel-Regel in `CLAUDE.md` :

"Use TDD and hexagonal architecture. Never delete files without stating why."

Folie 10 — Context Management: Qualität steuern

Was der Agent gerade "im Kopf" hat, ist entscheidend:

- **System Prompt** — Grundregeln (fix)
- **Tools Schema** — was er benutzen darf (read, edit, bash...)
- **Skills** — z. B. `code-review`, `explain-code`
- **Project-Files** — `CLAUDE.md`, `AGENTS.md`
- **Conversation History** — verwalten mit `/compact`, `/clear`, `/resume`
- `/context` — zeigt aktuellen Stand

Token sparen

1. Ballast raus (`/clear` , gezielte Files)
2. Zusammenfassen (`/compact "keep design decisions"`)
3. Nachhaltig lernen: *"Trag deine Erkenntnis in einer Zeile in `CLAUDE.md` ein"*

Folie 11 — Skills, Agents & MCP: Hebel mit Kontrolle

Skills — wiederverwendbare Commands

- **Knowledge-Skills:** Team-Standards, Architekturregeln oder Bibliotheks-Fallstricke wiederverwendbar machen.
- **Workflow-Skills:** wiederkehrende Abläufe wie Release Notes, Code Review oder SDD-Schritte standardisieren.
- **Tool-Skills:** dem Agent erklären, wie er Tools wie `gh`, `kubectl` oder Testkommandos korrekt nutzt.

Agents / Sub-Agents

- Copilot Reviewer läuft im Hintergrund, triggered nach File-Changes
- Diskussion offen: eigener Agent oder Sub-Agent? Trigger via Hook vs. Watch-Mode?

MCP (Model Context Protocol)

- Zugang für LLM zu externen Systemen (IntelliJ, Jira, DBs, GitHub)

- Erlaubt aktuelle Daten trotz Training-Cutoff

Folie 12 — Zwei Mini-Beispiele: SKILL.md und AGENTS.md

Damit die Präsentation greifbarer wird, zeigen wir je ein kleines Beispiel statt viele Details:

Beispiel 1 — SKILL.md : wiederkehrenden Prompt wiederverwenden

Use Case: Ein Team möchte wiederholt Code nach Raiffeisen-Standards reviewen lassen.

```
1  ---
2  name: raiffeisen-code-review
3  description: Prüft Code auf Qualität, Tests, Security und Raiffeisen-Kontext. Verwenden, wenn ein PR
4  oder Diff reviewed werden soll.
5  allowed-tools: Read, Grep, Glob
6  ---
7
8  Führe ein kurzes, strenges Code Review durch.
9  Prüfe insbesondere:
10 - keine Secrets, produktiven Daten oder vertraulichen Inhalte
11 - Tests und Quality Gates vorhanden
12 - klare Modulgrenzen und verständliche Namen
13 - keine unnötige Komplexität oder Scope-Erweiterung
14
15 Berichte Findings nach Priorität:
16 1. Kritisch – muss vor Merge gelöst werden
17 2. Warnung – sollte gelöst werden
18 3. Vorschlag – optional
```

Beispiel 2 — AGENTS.md : Projektregeln für den Agenten

Use Case: Ein Repo bekommt klare Leitplanken, damit der Agent nicht jedes Mal neu instruiert werden muss.

```
1  # AGENTS.md
2
3  ## Grundregeln
4  - Antworte und dokumentiere für dieses Repo auf Deutsch.
5  - Starte bei unklaren Anforderungen zuerst mit Rückfragen oder einem Plan.
6  - Schreibe keinen Code, bevor Scope, Datenklassifizierung und Akzeptanzkriterien klar sind.
7  - Keine Secrets, .env-Dateien, produktiven Daten oder vertraulichen Inhalte lesen oder ausgeben.
8
9  ## Engineering-Standards
10 - Bevorzuge einfache, wartbare Lösungen.
11 - Halte bestehende Architektur- und Modulgrenzen ein.
12 - Ergänze oder aktualisiere Tests für jede fachliche Änderung.
13 - Führe nach Änderungen die passenden lokalen Checks aus, z. B. Tests, Linting oder Build.
14
15 ## Review-Erwartung
16 - Liste am Ende geänderte Dateien, ausgeführte Checks und offene Risiken auf.
17 - Markiere Annahmen explizit, statt sie stillschweigend zu implementieren.
```

- ❗ Skills sind wiederverwendbare Mini-Prozesse. AGENTS.md ist die dauerhafte Projekt-Anleitung. Beides reduziert Prompt-Wiederholung und verhindert, dass wichtige Regeln nur im Chat-Verlauf stehen.

Slide 12 — Spec-driven development: make AI auditable

Nicht "prompt bis's passt", sondern strukturiert:

- 1 0. Clarify the feature and data classification
- 2 1. Write a small spec with acceptance criteria
- 3 2. Ask AI for questions and missing edge cases
- 4 3. Approve the plan before file changes
- 5 4. Implement in small steps
- 6 5. Verify with tests, review, and security checks

Loop: Spec → Review → Plan → Implement → Verify

(z. B. Shift-Tab-Mode "Echelon")

Kernpunkt: **Context Engineering** > Prompt Engineering.

Slide 13 — Nächste Schritte

- AI-Readiness-Score pro Repo (Slide 3 als Checkliste)
 - `CLAUDE.md` in unseren aktiven Repos anlegen
 - Erste `SKILL.md` Templates im Team-Space
 - Guardrails: `permissions-config.json` Team-Standard definieren
-

Slide 14 — Q & A

Merksatz:

Der AI-Agent ist so gut wie dein Repo, deine Tests und dein Context.
Alles andere ist Marketing.

TODO:

Context & Prompt Engineering

- Fundamentals — 5-min History, Context Window, Harness/LLM, Compaction, Dumb-Zone, Sub-Agents
- Token-Efficiency — practices & tools for reducing AI costs
- Project Memory — best way to build your `CLAUDE.md` (or `AGENTS.md`)
- Progressive Disclosure Pattern — skills, nested context, file references, tripwires

Agent Tools

- MCP — architecture, custom MCP example
 - Agent Tools — amazing things you can do via MCPs and CLIs (hands-on)
-

Spec-Driven Development

- When to directly prompt, plan-mode, brainstorming, or SDD?
- "Tres Amigos" requirements grilling, and handling business' vibe-coding
- Spec review — how to avoid fatigue-LGTM, skill erosion, and cognitive debt
- Develop a feature using OpenSpec

Agentic Workflows

- Multi-Agent Orchestration Patterns
- State-of-the-Art Agentic Workflow — practical take-aways
- Optimizing Code-Reviews — quality guardrails, multi-agent / multi-model review, guided human review
- Architecture Observability — drift-protected high signal/noise files

AI Security

- Prompt Injection Techniques
- Agent Supply-Chain Attacks — Skills, MCPs, library post-install scripts, RAG poisoning

Agent Sandboxing — permissions, auto-
